5. Construct a scheduling program with C that selects the waiting processwith the highest priority to execute next.

PROGRAM:

```
#include <stdio.h>

int main()

{

   int n, i, j;

   int bt[10], priority[10];

   int wt[10], tat[10];

   char process[10];

   int temp_bt, temp_priority;

   char temp_process;

   float avg_wt = 0, avg_tat = 0;

   printf("Enter the total number of Processes: ");

   scanf("%d", &n);

   if (n < 1 || n > 10)

   {

      printf("Invalid number of processes!\n");

      return 1;


   printf("\nPlease Enter the Burst Time and Priority of each process:\n");

   for (i = 0; i < n; i++)

   {

      process[i] = 'A' + i;

      printf("\nEnter the details of the process %c\n", process[i]);

      printf("Enter the burst time: ");

      scanf("%d", &bt[i])

      printf("Enter the priority: ");

      scanf("%d", &priority[i]);

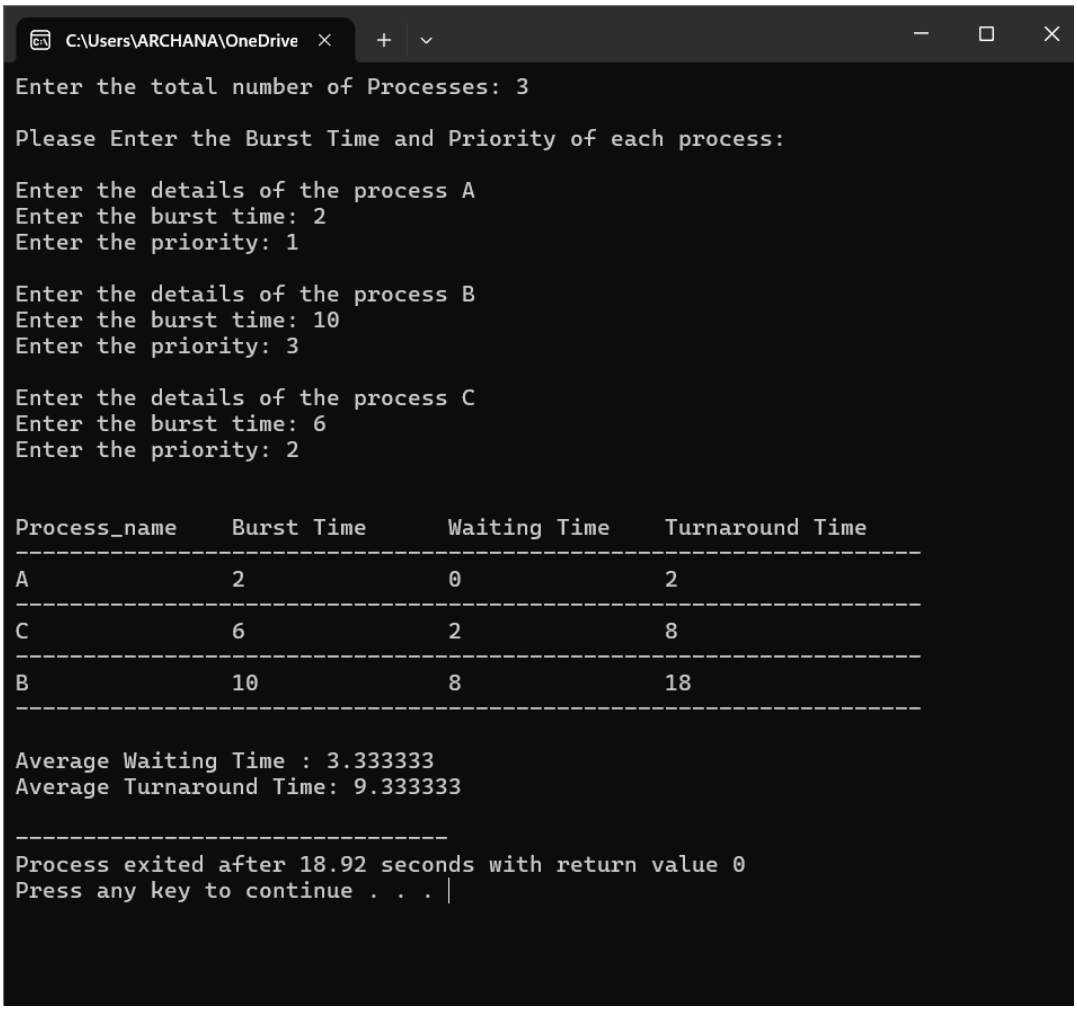
      if (bt[i] <= 0)

      {
```

```
        printf("Burst time must be positive!\n");
        return 1;
    }
}
/* Sort processes by priority:
   Higher priority = smaller priority number */
for (i = 0; i < n - 1; i++)
{
    for (j = i + 1; j < n; j++)
    {
        if (priority[i] > priority[j])
        {
            temp_priority = priority[i];
            priority[i] = priority[j];
            priority[j] = temp_priority;
            temp_bt = bt[i];
            bt[i] = bt[j];
            bt[j] = temp_bt;
            temp_process = process[i];
            process[i] = process[j];
            process[j] = temp_process;
        }
    }
}
wt[0] = 0;
tat[0] = bt[0];
for (i = 1; i < n; i++)
{
    wt[i] = wt[i - 1] + bt[i - 1];
    tat[i] = wt[i] + bt[i];
```

```
    }
    printf("\n\nProcess_name\tBurst Time\tWaiting Time\tTurnaround Time\n");
    printf("-------------------------------------------------------------------\n");
    for (i = 0; i < n; i++)
    {
        printf("%c\t\t%d\t\t%d\t\t%d\n",
               process[i], bt[i], wt[i], tat[i]);
        printf("-------------------------------------------------------------------\n");
        avg_wt += wt[i];
        avg_tat += tat[i];
    }
    printf("\nAverage Waiting Time : %f\n", avg_wt / n);
    printf("Average Turnaround Time: %f\n", avg_tat / n);
    return 0;
}
```

OUTPUT:



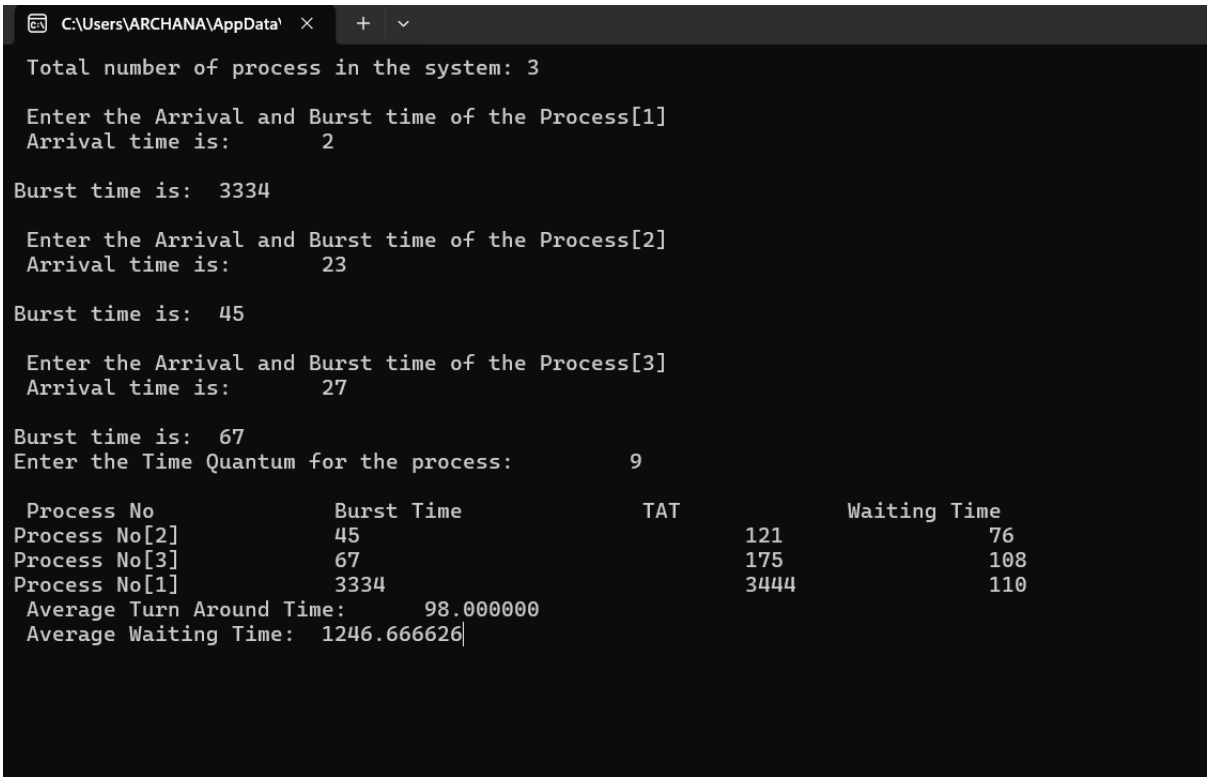
6. Construct a C program to simulate Round Robin scheduling algorithm with C.

PROGRAM:-

```
#include<stdio.h>
#include<conio.h>
int main()
{
int i, NOP, sum=0,count=0, y, quant, wt=0, tat=0, at[10], bt[10], temp[10];
float avg_wt, avg_tat;
printf(" Total number of process in the system: ");
scanf("%d", &NOP); y = NOP;
for(i=0; i<NOP; i++)
{
printf("\n Enter the Arrival and Burst time of the Process[%d]\n", i+1);
printf(" Arrival time is: \t");
scanf("%d", &at[i]);
printf(" \nBurst time is: \t");
scanf("%d", &bt[i]);
temp[i] = bt[i];
}
printf("Enter the Time Quantum for the process: \t");
scanf("%d", &quant);
printf("\n Process No \t\t Burst Time \t\t TAT \t\t Waiting Time ");
for(sum=0, i = 0; y!=0; )
{
if(temp[i] <= quant && temp[i] > 0)
{ sum = sum + temp[i]; temp[i] = 0; count=1; }
else if(temp[i] > 0)
{ temp[i] = temp[i] - quant;
sum = sum + quant;
}
```

```
if(temp[i]==0 && count==1)
{ y--;
printf("\nProcess No[%d] \t\t %d\t\t\t\t %d\t\t\t %d", i+1, bt[i], sum- at[i], sum-at[i]-bt[i]);
wt = wt+sum-at[i]-bt[i]; tat = tat+sum-at[i]; count =0;
} if(i==NOP-1)
{ i=0; } else if(at[i+1]<=sum)
{ i++; } else
{ i=0;
}
} avg_wt = wt * 1.0/NOP;
avg_tat = tat * 1.0/NOP;
printf("\n Average Turn Around Time: \t%f", avg_wt);
printf("\n Average Waiting Time: \t%f", avg_tat);
getch();
}
```

OUTPUT:



7. Construct a C program to implement non-preemptive SJF algorithm

PROGRAM:

```
#include<stdio.h>
int main()
{
int at[10],bt[10],pr[10];
int n,i,j,temp,time=0,count,over=0,sum_wait=0,sum_turnaround=0,start;
float avgwait,avgturn;
printf("Enter the number of processes\n");
scanf("%d",&n); for(i=0;i<n;i++)
{       printf("Enter the arrival time and execution time for process %d\n",i+1);
        scanf("%d%d",&at[i],&bt[i]);
        pr[i]=i+1;
}
for(i=0;i<n-1;i++)
{ for(j=i+1;j<n;j++)
{ if(at[i]>at[j])
{ temp=at[i];
at[i]=at[j];
at[j]=temp;
temp=bt[i];
bt[i]=bt[j];
bt[j]=temp;
temp=pr[i];
pr[i]=pr[j];
pr[j]=temp;
}
}
}
printf("\n\nProcess\t|Arrival time\t|Execution time\t|Start time\t|End time\t|waiting time\t|Turnaround time\n\n");
```

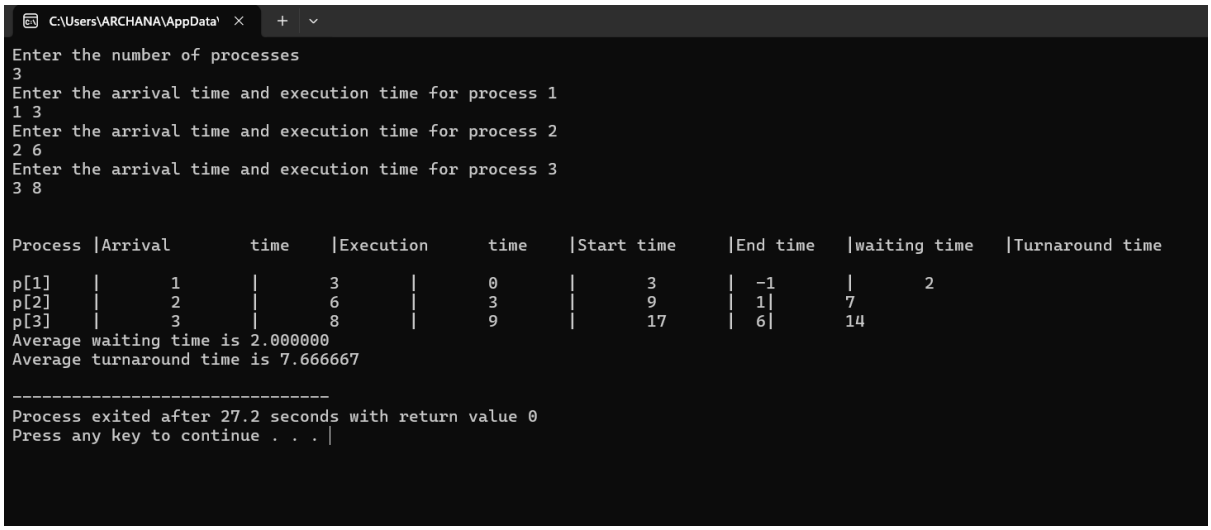
```
while(over<n)
{ count=0; for(i=over;i<n;i++)
{
if(at[i]<=time) count++;
else break;
} if(count>1)
{ for(i=over;i<over+count-1;i++)
{ for(j=i+1;j<over+count;j++)
{ if(bt[i]>bt[j])
{ temp=at[i];
at[i]=at[j];
at[j]=temp;
temp=bt[i];
bt[i]=bt[j];
bt[j]=temp;
temp=pr[i];
pr[i]=pr[j];
pr[j]=temp;
}
}
}
} start=time;
time+=bt[over];
printf("p[%d]\t|\t%d\t|\t%d\t|\t%d\t|\t%d\t|\t%d\t|\t%d\n",pr[over],
at[over],bt[over],start,time,time-at[over]- bt[over],time-at[over]);
sum_wait+=time-at[over]-bt[over];
sum_turnaround+=time-at[over]; over++;
}
avgwait=(float)sum_wait/(float)n;
avgturn=(float)sum_turnaround/(float)n;
printf("Average waiting time is %f\n",avgwait);
```

printf("Average turnaround time is %f\n",avgturn);

return 0;

}

OUTPUT



8. Construct a C program to simulate Round Robin scheduling algorithm with C.

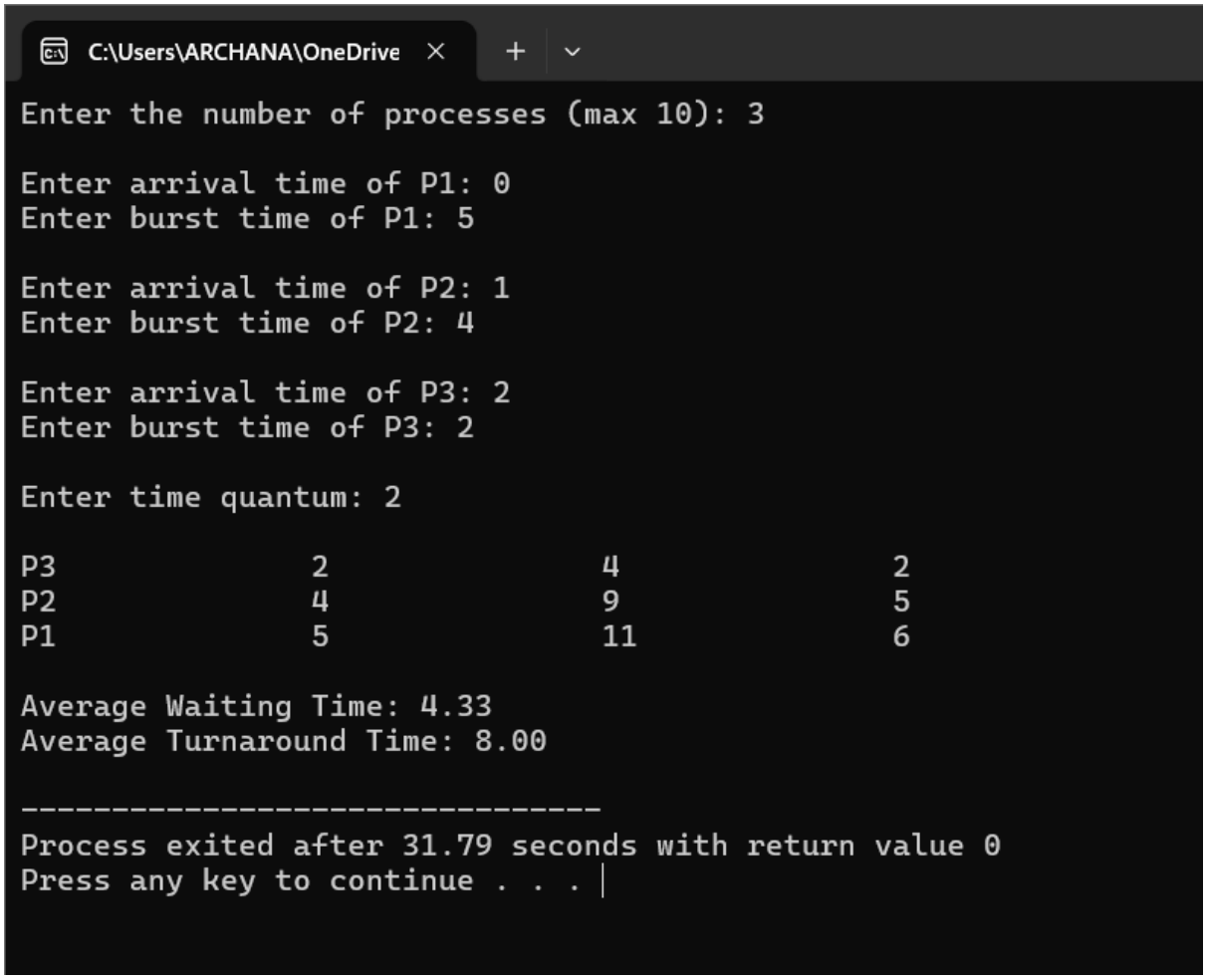
PROGRAM:

```
#include <stdio.h>
int main()
{
   int i, n, quantum;
   int time = 0, completed = 0;
   int at[10], bt[10], remaining[10];
   int wt[10] = {0}, tat[10] = {0};
   int visited[10] = {0};
   float avg_wt = 0, avg_tat = 0;
   printf("Enter the number of processes (max 10): ");
   scanf("%d", &n)
   if (n < 1 || n > 10)
   {
      printf("Invalid number of processes!\n");
```

```
    return 1;
}
for (i = 0; i < n; i++)
{
    printf("\nEnter arrival time of P%d: ", i + 1);
    scanf("%d", &at[i]);
    printf("Enter burst time of P%d: ", i + 1);
    scanf("%d", &bt[i]);
    if (at[i] < 0 || bt[i] <= 0)
    {
        printf("Invalid arrival or burst time!\n");
        return 1;
    }
    remaining[i] = bt[i];
}
printf("\nEnter time quantum: ");
scanf("%d", &quantum);
if (quantum <= 0)
{
    printf("Time quantum must be positive!\n");
    return 1;
}
while (completed < n)
{
    int executed = 0;
    for (i = 0; i < n; i++)
    {
        if (remaining[i] > 0 && at[i] <= time)
        {
            executed = 1;
```

```
            if (remaining[i] > quantum)
            {
               time += quantum;
               remaining[i] -= quantum;
            }
            else
            {
               time += remaining[i];
               remaining[i] = 0;
               completed++;
               tat[i] = time - at[i];
               wt[i] = tat[i] - bt[i];
               printf("\nP%d\t\t%d\t\t%d\t\t%d",
                     i + 1, bt[i], tat[i], wt[i])
               avg_wt += wt[i];
               avg_tat += tat[i];
            }
         }
      }
      if (!executed)
      {
         time++;
      }
   }
   avg_wt = avg_wt / n;
   avg_tat = avg_tat / n;
   printf("\n\nAverage Waiting Time: %.2f", avg_wt);
   printf("\nAverage Turnaround Time: %.2f\n", avg_tat);
   return 0;
}
```

OUTPUT:


C:\Users\ARCHANA\OneDrive
Enter the number of processes (max 10): 3
Enter arrival time of P1: 0
Enter burst time of P1: 5
Enter arrival time of P2: 1
Enter burst time of P2: 4
Enter arrival time of P3: 2
Enter burst time of P3: 2
Enter time quantum: 2
P3 2 4 2
P2 4 9 5
P1 5 11 6
Average Waiting Time: 4.33
Average Turnaround Time: 8.00
Process exited after 31.79 seconds with return value 0
Press any key to continue . . .